Fully-Decentralized Federated Learning for QoE Estimation

Minh-Duc NGUYEN†, Van TONG†, Sami SOUIHI*, and Abdelhamid MELLOUK*
*LISSI-TincNET Research Team, University of Paris-Est Creteil, France
Email: (sami.souihi and mellouk)@u-pec.fr
† School of Information and Communication Technology, Hanoi University of Science and Technology, Vietnam
Email: duc.nm183713@sis.hust.edu.vn, (vantv and anhth)@soict.hust.edu.vn

***Abstract*—In the past, Quality of Service (QoS) was taken into account to evaluate the performance of multimedia services (e.g., video streaming, file transfer, etc.). However, it cannot reflect the user's perception, which is considered a crucial consideration by these services nowadays. Therefore, the emergence of Quality of Experience (QoE) is a potential solution. QoE can be measured via many parameters provided by Internet Service Providers (ISP), Application Service Providers (ASP), or end-users. However, privacy concerns hinder data sharing between the parties involved.**

To address these limitations, this paper proposes a QoE estimation mechanism that leverages Federated Learning. This mechanism aims to guarantee data privacy when no party needs to disclose their data to others. Moreover, the proposed mechanism incorporates the concept of a Decentralized Autonomous Organization (DAO) to mitigate the risk of a single point of failure in the centralized architecture of Federated Learning. It enables all participants to evaluate and select the model efficiently. The experimental results illustrate that the proposal surpasses the centralized solutions and guarantees data privacy.

I. Introduction

In the past, ISPs (Internet Service Providers) and ASPs (Application Service Providers) took into account Quality of Service (QoS) to evaluate network performance. This metric was calculated by different parameters, including bandwidth, latency, jitter, and so on. Nowadays, the user's perception is put a special focus on by multimedia services, but QoS metrics are not enough to reflect it. The emergence of Quality of Experience (QoE) is a potential solution to overcome this limitation. QoE refers to the user perception of multimedia services, including video streaming, file transfer, and so forth. It takes into account the network parameter as well as other factors such as human factors (e.g., gender, age, mood, etc.) and device-related factors (e.g., screen resolution, display size, etc.) [1]. As a result, QoE can be measured independently by ISP, ASP, or end-users. According to recent related work, there are three QoE estimation methods: subjective, objective, and hybrid.

- **Subjective:** This method requires end-users to directly evaluate their perceptions about multimedia services (e.g., through watching a video, etc.), so it represents user satisfaction. However, this method is expensive and ineffective for real-time QoE monitoring due to human intervention.
- **Objective:** It builds an objective model to estimate the user perception using network parameters, application parameters, and so on. Although this method does not require human intervention, selecting an appropriate objective model is sometimes complicated.
- **Hybrid:** The final one lies between the subjective and objective approaches, strengthening these two approaches. This mechanism uses a subjective dataset to build a QoE model estimation model using Machine Learning algorithms.

In this work, we take into account the hybrid method, which is studied extensively by the research community. QoE relies on many factors, which can be provided by many different parties (ISP, ASP, or end-users). However, these parties are unwilling to share data with others due to security concerns. Therefore, Google designed and studied Federated Learning (FL) extensively in 2016 to solve this limitation [2], [3]. It not only guarantees the data privacy of the parties but also leverages data from different parties for enhancing QoE estimation. This technique allows each party to train the model separately with their datasets instead of combining all the data with training the model as in traditional solutions. The individual models are then contributed to a central server to aggregate into a global model with improved accuracy. Despite its benefits, FL suffers from several limitations, as follows:

- **Single point of failure:** Traditional FL-based solutions require a central server, known as an aggregator, to aggregate local models into a global model. If this server is overloaded (e.g., due to a DDoS attack, etc.), the entire solution will be influenced.
- **Malicious clients and false data:** We cannot guarantee that all clients are trusted. Malicious clients may provide inaccurate information about their local training results. As a result, the erroneous data will negatively impact the performance of the global model.
- **Lack of incentive mechanism:** Model training requires a lot of computational resources, but there is no encouragement for the participants. Moreover, attackers who update inaccurate models are not subject to penalty. As a result, people lose interest in contributing to the FL-based solutions.

The above shortcomings reduce the efficiency and reliability of FL. Therefore, Blockchain is studied to solve these limitations by its characteristics such as decentralization, traceability, consensus, security, immutability, and so on [2]. First, the central aggregator can be replaced by a peer-to-peer Blockchain network, and the nodes in this network can aggregate local models, avoiding the unreliability of the central server. Furthermore, Blockchain provides a transaction verification mechanism in which untrusted clients or malicious

models can be verified and rejected before the global model is aggregated. Finally, Blockchain's smart contract can be used to create mechanisms that reward trustworthy contributors and block suspicious clients from attempting to compromise the system.

Outline: The remainder of the paper is structured as follows: SectionII presents some related work and background around Federated Learning and Blockchain. Then, the FL-based QoE estimation mechanism using Blockchain is described in Section III. Section IV shows the experimental results of the proposed mechanism. Finally, the paper concludes with Section V, highlighting future works.

II. Background and Related Work

This section presents background and recent studies on FL-based QoE estimation and Blockchain-based FL architectures. Besides, we describe the motivation to build a fully decentralized FL architecture.

A. Why Federated Learning?

The explosive growth of Artificial Intelligence (AI) in recent years has raised concerns about data privacy. We all know that machine learning models need to be trained with a huge amount of data to obtain high accuracy. One solution is to collect private and sensitive data from users illegally to obtain enough data for their machine-learning models. Federated Learning (FL) is a technique introduced by Google 2016 to solve this problem. The main idea behind this mechanism is that clients (e.g., user's devices, etc.) train the model on their local machine with their data and then send these local models to a centralized server for aggregation without requiring clients to send data to a server for model training. This helps to ensure data privacy since only the model is sent over the network, and the data remains confidential to the clients. Furthermore, bandwidth consumption is also reduced because the model size is significantly smaller than the data size. FL is categorized into two approaches: *Horizontal* and *Vertical*.

- **Horizontal Federated Learning (HFL):** This approach is used where the dataset from each client has the same features.
- **Vertical Federated Learning (VFL):** Unlike Horizontal FL, each client's dataset has samples on a group of users but different features between them.

VFL requires each participant to obtain different features on a group of users, so it is challenging in the context of QoE estimating. Therefore, in this paper, HFL is taken into account. The HFL comprises millions of devices of different end-users, and each dataset contains the same feature sets (e.g., age, gender, education, etc.). The detail of HFL is depicted as Fig. 1:

1) A central server (an aggregator) initializes a global Machine Learning model for all clients and an initial set of weights.
2) Server sends the global model to clients.
3) Each client trains the model with its data, resulting in a local model.
4) The client sends back its local model to the server.
5) The server aggregates local models into a new global model using aggregation algorithms such as FedAvg, FedAdam, and so forth.
6) Continue repeating step 2.

This implementation requires a central server to aggregate the local models, resulting in the Single Point of Failure problem. In addition, a model verification mechanism should also be carefully designed to ensure that no malicious models are involved in the aggregation.

There are many existing studies for FL-based QoE estimation. Gao et al. [4] proposed a Deep Learning-based QoE model that can extract personalized characteristics from massive multimedia data. Besides, an FL architecture is proposed to solve the data sparsity problem and ensure privacy. Although this architecture can avoid a single point of failure, this architecture does not ensure that each aggregated model vector is reliable. On the other hand, the author also does not provide a method to collect models between independent users, and model exchange between peers can be costly.

B. Blockchain-based Federated Learning

Blockchain is a distributed ledger managed by a peer-to-peer network. Its data structure is a list of records (blocks) securely linked together by a cryptographic hash function. Each block consists of a header containing information such as timestamps, the hash of the previous block, and a body containing transaction data. Each node in the p2p network includes a copy of the ledger. When a new block is added, the nodes must validate and approve it through consensus mechanisms. Besides, the data written to the Blockchain is stored permanently and immutably because a tiny change in a block will invalidate the hash of that block and the following blocks. The changed data will be overwritten with the old data by creating a transaction and adding a new block. However, the old values are still permanently stored on the previous blocks and can be retrieved in history easily.

The Single Point of Failure problem of Federated Learning can be solved by replacing the aggregator with Blockchain, as it is a fault-tolerant p2p network. When a node has a situation such as an attack, the remaining nodes still contain a copy of the ledger and may be ready to provide services. On the other hand, in the HFL system, the clients all have the same role. This is similar to the peer in the Blockchain network. Therefore, all clients can participate in the evaluation and vote for local models by saving the voting results transparently on the Blockchain through the Smart Contract feature. This reduces subjectivity compared to only evaluating the model on a single centralized server like the traditional FL architecture. Several recent studies have also proposed different architectures for implementing FL on Blockchain.

Pradip et al. [5] and Li et al. [6] present Blockchain architectures specific to FL, where each block's body stores model updates while the block header includes metadata. Cui et al. [7] design Smart Contracts to hold model updates in transactions. Most Blockchain-based FL (BCFL) solutions focus on storing the updated models on the Blockchain by implementing specialized Blockchain networks with a unique block structure or

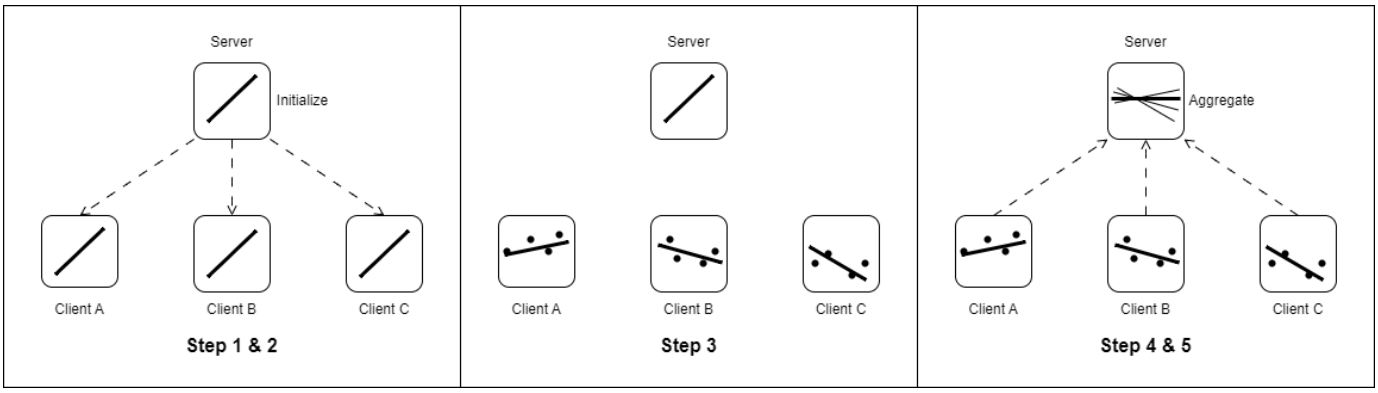

Fig. 1: Horizontal Federated Learning

storing in the transactions of well-known public Blockchains. However, implementing a dedicated Blockchain is complicated and expensive to store updates. Besides, it isn't easy to scale for other purposes. However, with a public Blockchain, a transaction's gas cost will rise with its size, making saving model updates to the transaction expensive.

The verification mechanisms to decide which model updates can be used for aggregation are emphasized in recent BCFL studies. Most approaches evaluate model updates using testing datasets to calculate accuracy and determine whether the model is good or bad based on a predetermined accuracy threshold. However, each approach will have a different implementation. Jiawen et al. [8], the task publisher, provides the testing dataset stored on the Blockchain. Then, miners will verify model updates by that dataset. Using public datasets on the Blockchain for testing will make the updated evaluation results subjective, but preparing new test datasets for each round is not a simple task. It would be more objective for each client to download the updates, evaluate themselves using their data, and then vote for the updates which are believed as good. This method requires a consensus mechanism between clients. Cui et al. [7] proposed a mechanism that required randomly selected members to vote on whether or not the updates were reliable; however, it is difficult to demonstrate how this avoids the influence of subjectivity or ensures security when the majority of randomly selected members may be malicious clients.

III. Fully-Decentralized Federated Learning Mechanism for QoE Estimation

This section presents the Deep Learning-based QoE estimation model and the fully decentralized FL architecture, which contains three main phases: storage, validation, and aggregation. In the storage phase, we suggest an effective decentralized storage mechanism rather than centralized server storage, as in traditional FL architectures. Next, the verification phase describes how clients choose reliable updates. Finally, a new global model will be aggregated in the aggregation phase from verified updates in the previous phase. The proposed mechanism is described in Fig. 2.

1) A client uploads QoE Model Update to IPFS and gets back a unique CID of that file.
2) Client sends the received CID to DAO Smart Contract, then it will trigger an event to notify other clients that a new model is available.
3) Other clients download Model Update from IPFS by CID stored in the smart contract.
4) Each client evaluates that update with their dataset.
5) Each client votes to accept or reject the update based on the evaluation result.
6) Smart contract selects reliable models based on voting results.
7) Each client self-aggregates selected models on their local machine.

A. Storage

In a conventional FL architecture, local model updates are uploaded to a centralized server, leading to the risk of data modification due to cybersecurity attacks. Blockchain can solve this problem, as any data stored on the Blockchain is permanent and immutable. Therefore, we propose using InterPlanetary File System (IPFS) to keep model updates instead of directly storing them on Blockchain to implement a BCFL cost-effectively.

The InterPlanetary File System comprises composable peer-to-peer protocols for addressing, routing, and transferring content-addressed data in a decentralized file system. IPFS aims to store data permanently, which is difficult to modify. A file will be cryptographically hashed before adding to IPFS and assigning a Content Identifier (CID). Everyone who knows the CID can view and download this distinct fingerprint. Data on IPFS are resistant to alteration since any modifications result in a new version and obtaining a new CID instead of overwriting the original data. The difference between IPFS and Blockchain is that a node in IPFS only stores a part of data instead of complete data as in Blockchain. When a node queries a file, it will query other peer nodes by CID. After downloading the file, the node caches this copy and becomes its new provider until the file is removed from the node's cache. There is a garbage collection mechanism, which means that IPFS nodes will periodically delete some old data to increase available space for new data. Nevertheless, IPFS offers opportunities to store data permanently. In the FL architecture, participants

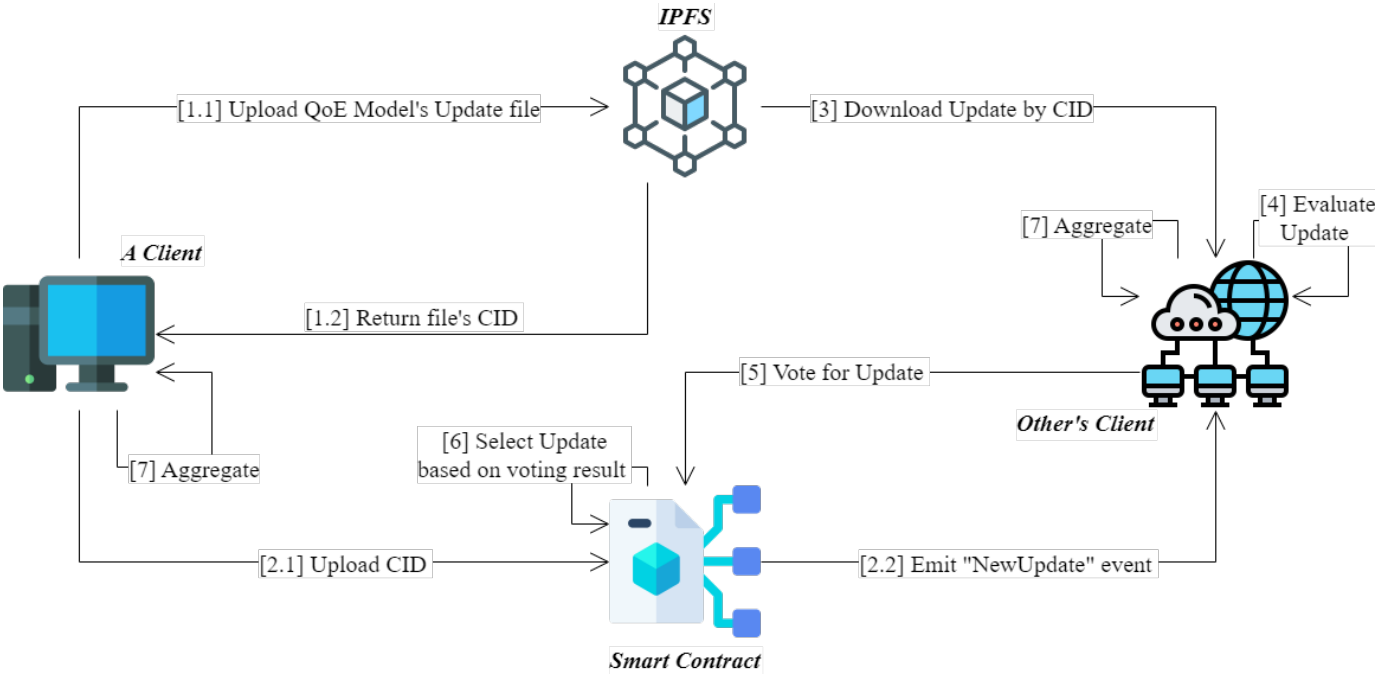

Fig. 2: Blockchain-based FL mechanism for QoE.

focus on the recent model upgrades instead of the updates in the early stage. Hence, IPFS is a potential solution and more appropriate than Blockchain.

As in Fig. 2, when a client (e.g., ISP, ASP, user, etc.) wants to contribute an existing model to the proposed mechanism, the client first uploads the model to IPFS and receives a CID, then sends this CID to a smart contract to store. Then, the client waits for other clients to evaluate in the following phases. Everyone who knows the CID can easily download and access the model from their local machines.

The processing time between various clients is different, so the clients will send the model updates asynchronously to the smart contract. Then, the smart contract will notify others so that other clients can know and evaluate.

B. Verification

In this section, we propose a voting mechanism leveraging the concept of a Decentralized Autonomous Organization (DAO). DAO is a general-purpose organization that operates under rules written as computer programs and keeps on the Blockchain as smart contracts. DAOs are decentralized communities where members collectively own and govern the organization, eliminating the necessity for centralized leadership. [9]

The DAO contract stores the contribution point p_k of the client k. The client (e.g., ISPs, ASPs, users, etc.) gains contribution points when they contribute to the system by sending cryptocurrency to a smart contract or uploading accurate model updates approved by others. In other words, uploading inaccurate model updates disapproved by others results in decreasing contribution points. For each model update u, clients can give a single *Accept*, *Reject*, or *Abstain* ballot. After a period of time τ, the decision whether the model update u should be included in the aggregation is automatically made by the smart contract based on the transparent voting results. This process is described in Alg. 1.

Algorithm 1 Model's Update Selection

For each model update m: A, R, and P represent the set of clients that accept, reject, and abstain, respectively

$AcceptPoint \leftarrow \sum_{k \in A} p_k$
$RejectPoint \leftarrow \sum_{k \in R} p_k$
$AbstainPoint \leftarrow \sum_{k \in P} p_k$
$TotalPoint \leftarrow \sum p_k$
$Participation \leftarrow \frac{AcceptPoint + RejectPoint + AbstainPoint}{TotalPoint}$
$AcceptRate \leftarrow \frac{AcceptPoint}{AcceptPoint + RejectPoint}$
if $Participation \geq \sigma$ **then** *// Valid voting result*
    if $AcceptRate \geq \varrho$ **then**
        $S \leftarrow S \cap u$

A ballot of client k has a weight that is equal to the client's contribution point p_k. At the end of the poll period, the vote's result is only considered valid if the total weight of ballots exceeds quorum σ. Suppose the ratio between the *Accept* weight and the sum of *Accept* and *Reject* weights is bigger than ϱ. In that case, it will be added to the set S containing the model updates that are considered reliable for aggregation.

C. Aggregation

After a fixed amount of updates is selected, clients will enter the final aggregation phase. Unlike the standard FL system, where updates are aggregated at the server and then returned to a new global model for clients, in BCFL, Smart Contracts maintain a state of the most recent model updates, which

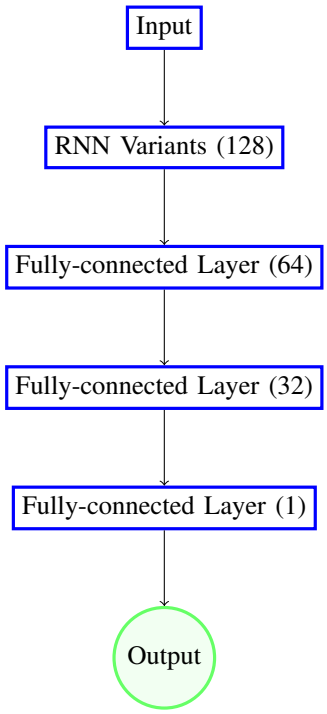

Fig. 3: QoE estimation model

clients will download and aggregate on their local computers. Furthermore, rather than receiving only the server's predefined algorithm results, each client can aggregate according to different algorithms they choose, such as FedAvg, FedAdam, FedYogi, FedAdagrad, and so on [3].

In this paper, we consider FedAvg in the proposed mechanism because it is studied extensively by the research community [3], [10]. The weight update w_{i+1} of this algorithm in round $i+1$ is described as in Equ. 1:

$$w_{i+1} = \sum \frac{n_k}{m_i} w_{i+1}^k \tag{1}$$

where k is the index of the client, n_k is the number of samples used for training, w_{i+1}^k is the weight of model update from client k and $m_i = \sum n_k$.

D. QoE estimation model

Deep Learning models have gained popularity for their strong ability to extract implicit features from data. Among various Deep Learning algorithms, Recurrent Neural Network (RNN) contains recurrent connections between neurons, enabling them to retain information from previous steps in a sequence. Therefore, RNN is an appropriate algorithm for QoE estimation because the user perception relies on the prior information. However, RNN suffers from gradient vanishing, hindering effective learning. Furthermore, the limited memory of RNNs makes them prone to forgetting previously acquired knowledge. This paper considers advanced variants (LSTM, GRU, and Bidirectional LSTM) to address these issues. The detail of the QoE estimation model is described in Fig. 3.

QoE estimation contains two main parts: feature extraction and classification. The former considers the input and extracts the implicit features using RNN variants (LSTM, GRU, and Bidirectional LSTM). In the latter, these implicit features are fed into the classification, composed of three fully-connected layers to estimate the quality level from 1 to 5. The quality level is the user's perception corresponding to the input.

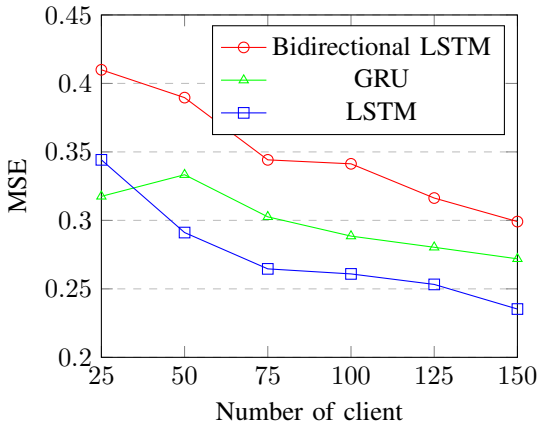

Fig. 4: MSE of different Deep Learning algorithms for QoE estimation.

IV. Experiment Results

A. Experimental setup

To evaluate the performance of the proposed mechanism, we build a testbed and use the Poqemon dataset [11] thoroughly investigate different Deep Learning algorithms.

This dataset contains various QoE Influence Factors (QoE IFs) collected using a VLC media player. It also includes the subjective Mean Opinion Score (MOS), the most widely used metric to represent QoE. The MOS is usually a single rational number between 1 and 5, with one being the lowest and 5 being the highest perceived quality. Categories of QoE IFs in this dataset consist of:

- Video parameters from VLC video player
- Network information
- Device characteristic
- User's profile
- User's feedback

The dataset is split into 180 subsets D by $user\_id$, of which the initialization part contains 15 subsets used for model initialization. The remaining 165 subsets are divided into two parts: 150 subsets for training and 15 last subsets for testing. In the training part, 150 subsets are divided into six parts corresponding to six rounds. In this paper, we consider different DL algorithms, and each one is implemented as follows: First, we initialize a global model with different weights w_0 with each RNN's variant. Then the following steps are repeated several times to calculate the MSE values of the global models aggregated after each round i of FL:

1) Train the model from weights w_i separately with 25 subsets from D_{25i+1} to $D_{25(i+1)}$, result is 25 local models.
2) Aggregate that 25 models by FedAvg algorithm to a new global model which has weights w_{i+1}.

3) Evaluate the new global model in step 2 with the test dataset.

After evaluating the performance of different Deep Learning algorithms, we select an appropriate one to evaluate the performance between federated and centralized learning solutions. First, we initialize the DL model with the same weights w_0 for Centralized Learning. Then the following steps are repeated several times to compare the MSE of the model trained by FL in each round with the MSE of the traditionally trained model with the same amount of data. For each round i:

1) Train the model from weights w_0 with $25i$ subsets from D_1 to D_{25i}.
2) Evaluate the new model with the test dataset.

B. Performance Analysis

Fig. 4 illustrates the Mean Squared Error (MSE) of different learning algorithms for QoE estimation. MSE of all models decreases as the number of clients increases, which means the accuracy increases as more data are used. GRU is more accurate than LSTM in the first round but worse in all subsequent rounds. The reason is that GRU is a simple version of LSTM. Concretely, LSTM is composed of more gates and parameters than GRU, so it leads to more flexibility and expressiveness compared to GRU. A noticeable feature from Fig. 4 is that the MSE of Bidirectional LSTM is higher than the figure for LSTM. Bidirectional LSTM contains one more LSTM layer in comparison with the original LSTM, so Bidirectional LSTM can reverse the direction of information flow. Bidirectional LSTM is effective for natural language processing, but we only need to learn the relationship between different features of a sample in QoE estimation. Therefore, its MSE is not as good as the figure for the LSTM algorithm. The performance of the LSTM algorithm is higher than the others, so it is chosen for the QoE estimation.

Fig. 5 shows the MSE of LSTM with classic centralized learning (CL) and LSTM with federated learning (FL) to evaluate their performances. We combined all the data for each quantity of clients and trained the first initialized model rather than preparing and aggregating in each round as FL. The MSE of FL is nearly equal to the figure for CL, with approximately 0.28. Besides, the MSE of both mechanisms decreases slightly when the number of clients increases.

V. Conclusion

In this work, we proposed a Blockchain-based Federated Learning mechanism for QoE estimation, which ensures data privacy, origin, transparency, and integrity of the model and keeps the accuracy close to traditional centralized learning models. First, we propose Smart Contracts following the structure of a Decentralized Autonomous Organization, which allows participants to vote for model updates and use a contribution-based vote power mechanism to ensure objectivity and safety for the voting result. Then, we compare the performance of several Deep Learning algorithms with a specific dataset to choose the appropriate model for QoE estimation. Finally, we evaluate the performance of the proposed mechanism and the centralized solution related to Mean Square Error.

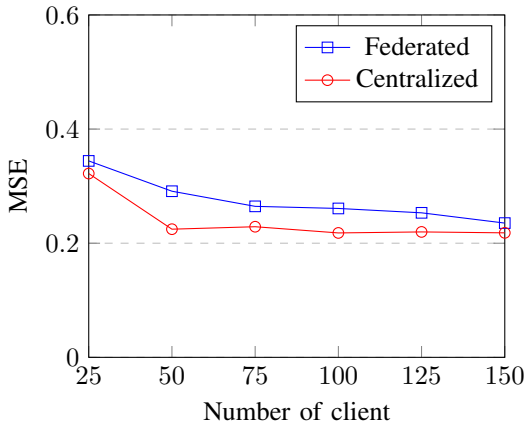

Fig. 5: Comparison of MSE between the proposal and the centralized solution for QoE estimation.

In the future, we will evaluate the proposal with different aggregation algorithms to select an appropriate one. Besides, we will consider other datasets to thoroughly assess the proposal's performance.

References

[1] U. Reiter, K. Brunnström, K. De Moor, M.-C. Larabi, M. Pereira, A. Pinheiro, J. You, and A. Zgank, *Factors Influencing Quality of Experience*. Cham: Springer International Publishing, 2014, pp. 55–72. [Online]. Available: https://doi.org/10.1007/978-3-319-02681-7_4

[2] Z. Wang and Q. Hu, "Blockchain-based federated learning: A comprehensive survey," 2021.

[3] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," 2023.

[4] Y. Gao, X. Wei, and L. Zhou, "Personalized qoe improvement for networking video service," *IEEE Journal on Selected Areas in Communications*, vol. 38, no. 10, pp. 2311–2323, 2020.

[5] P. K. Sharma, J. H. Park, and K. Cho, "Blockchain and federated learning-based distributed computing defence framework for sustainable society," *Sustainable Cities and Society*, vol. 59, p. 102220, 2020. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2210670720302079

[6] Y. Li, C. Chen, N. Liu, H. Huang, Z. Zheng, and Q. Yan, "A blockchain-based decentralized federated learning framework with committee consensus," *IEEE Network*, vol. 35, no. 1, pp. 234–241, 2021.

[7] L. Cui, X. Su, Z. Ming, Z. Chen, S. Yang, Y. Zhou, and W. Xiao, "Creat: Blockchain-assisted compression algorithm of federated learning for content caching in edge computing," *IEEE Internet of Things Journal*, vol. 9, no. 16, pp. 14 151–14 161, 2022.

[8] J. Kang, Z. Xiong, C. Jiang, Y. Liu, S. Guo, Y. Zhang, D. Niyato, C. Leung, and C. Miao, "Scalable and communication-efficient decentralized federated edge learning with multi-blockchain framework," 2020.

[9] Y. El Faqir, J. Arroyo, and S. Hassan, "An overview of decentralized autonomous organizations on the blockchain," in *Proceedings of the 16th International Symposium on Open Collaboration*, ser. OpenSym '20. New York, NY, USA: Association for Computing Machinery, 2020. [Online]. Available: https://doi.org/10.1145/3412569.3412579

[10] C. Zhang, Y. Xie, H. Bai, B. Yu, W. Li, and Y. Gao, "A survey on federated learning," *Knowledge-Based Systems*, vol. 216, p. 106775, 2021. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0950705121000381

[11] L. Amour, S. Souihi, and A. Mellouk, "Poqemon-qoe-dataset," https://github.com/Lamyne/Poqemon-QoE-Dataset, 2020.